Streams

Pipelines that filter, transform, and summarize data

Much of the code you write loops over a collection to find, change, count, or group its elements. A *stream* does the same work as a *pipeline* of steps, each one a method that takes a lambda, and the result often reads like a description of the task. This part covers how pipelines are built, the operations used most, `Optional`, the collectors that group and summarize, and the rules and pitfalls that catch people. It assumes the lambda expressions notes. For reading, see Core Java, Volume II, Chapter 1. In class, the Speedrun Leaderboard example uses streams in Checkpoints 5 and 6.

A note about these notes. I wrote these notes with help from AI, from the examples and explanations I use in class. The AI drafted much of the text and code, and I review and revise them. Every example was compiled and run, and every output and error message shown is real; long stack traces are shortened. If something looks wrong, does not match what you see in Eclipse, or contradicts the book, tell me. The textbook is the tiebreaker, and the compiler beats us both. These notes do not replace the assigned chapters.

1. The Examples

The examples use one small record and a playlist of seven songs. The songs and artists are invented.

```
public record Song(String title, String artist, int seconds) { }
```

Table 1: The playlist used in the examples, in its original order.

Title	Artist	Seconds
Midnight Drive	Neon Harbor	254
Paper Moons	The Lanterns	198
Slow Burn	Neon Harbor	312
Northern Lights	Ada Vale	287
Echo Park	The Lanterns	221
Glass Garden	Ada Vale	345
Static	Neon Harbor	176

`Playlist.songs()` returns these songs as a `List<Song>` that cannot be changed.

2. Loops and Pipelines

Here is one task done twice: the titles of the songs longer than four minutes, in alphabetical order.

```
List<String> loop = new ArrayList<>();
for (Song s : songs) {
    if (s.seconds() > 240) {
        loop.add(s.title());
```

```
    }
}
Collections.sort(loop);

List<String> stream = songs.stream()
        .filter(s -> s.seconds() > 240)
        .map(Song::title)
        .sorted()
        .toList();
```

```
[Glass Garden, Midnight Drive, Northern Lights, Slow Burn]
[Glass Garden, Midnight Drive, Northern Lights, Slow Burn]
```

Both give the same list. The loop says *how* to build it, one step at a time. The pipeline says *what* the result is: the songs, keeping the long ones, as titles, sorted, as a list.

3. The Parts of a Pipeline

Every pipeline has three parts:

1. **A source** that produces elements, such as `songs.stream()`.
2. **Intermediate operations**, such as `filter`, `map`, and `sorted`. Each returns a new stream, so they can be chained.
3. **One terminal operation**, such as `toList`, `count`, or `forEach`, which produces the result and ends the stream.

A stream does not store anything, and it never changes its source: after the pipeline above, `songs` still holds all seven songs, in the same order.

4. Creating Streams

Every collection has a `stream()` method. A few other sources are worth knowing:

```
System.out.println(Stream.of("Static", "Echo Park", "Slow Burn").count());
int[] ratings = {4, 5, 3, 5};
System.out.println(Arrays.stream(ratings).sum());
System.out.println(IntStream.rangeClosed(1, 5).sum());
```

```
3
17
15
```

`Stream.of` makes a stream of the values given. `Arrays.stream` makes one from an array. `IntStream.rangeClosed(1, 5)` produces the numbers 1 through 5, and `range(1, 5)` would stop before 5.

5. Intermediate Operations

Table 2: Common intermediate operations.

Operation	Result
`filter(predicate)`	Only the elements the predicate accepts
`map(function)`	Each element replaced by what the function returns
`sorted()`	The elements in natural order; `sorted(comparator)` for any other order
`distinct()`	The elements with duplicates removed, using `equals`
`limit(n)`	At most the first `n` elements
`skip(n)`	All but the first `n` elements

```
System.out.println(songs.stream().map(Song::artist).distinct().toList());
System.out.println(songs.stream().map(Song::title).sorted().limit(3).toList());
System.out.println(songs.stream().map(Song::title).sorted().skip(5).toList());
```

```
[Neon Harbor, The Lanterns, Ada Vale]
[Echo Park, Glass Garden, Midnight Drive]
[Slow Burn, Static]
```

`map` can change the type of the stream: `songs.stream()` is a `Stream<Song>`, and after `map(Song::artist)` it is a `Stream<String>`.

6. Terminal Operations

```
System.out.println(songs.stream().anyMatch(s -> s.seconds() > 340));
System.out.println(songs.stream().allMatch(s -> s.seconds() > 180));
System.out.println(songs.stream().noneMatch(s -> s.artist().isBlank()));

Optional<Song> longest = songs.stream().max(Comparator.comparingInt(Song::seconds));
System.out.println(longest.map(Song::title).orElse("no songs"));

int total = songs.stream().map(Song::seconds).reduce(0, Integer::sum);
System.out.println(total);
```

```
true
false
true
Glass Garden
1793
```

- `anyMatch`, `allMatch`, and `noneMatch` answer a yes-or-no question about the elements. Static, at 176 seconds, is why `allMatch` is false.
- `min` and `max` take a comparator and return an `Optional`, explained next.

- `reduce(0, Integer::sum)` combines the elements into one value: it starts with 0 and adds each element in turn. Section 8 shows a simpler way to add numbers.
- `count`, `toList`, `forEach`, `findFirst`, and `collect` are terminal operations too.

7. Optional

`max`, `min`, and `findFirst` might have nothing to return, because the stream might be empty. Rather than return `null`, they return an `Optional`: a box that holds either one value or nothing.

```
Optional<Song> byNobody = songs.stream()
        .filter(s -> s.artist().equals("Nobody"))
        .findFirst();
System.out.println(byNobody);
System.out.println(byNobody.map(Song::title).orElse("no songs"));
byNobody.ifPresent(s -> System.out.println("found " + s.title()));
```

```
Optional.empty
no songs
```

- `orElse(value)` returns the contents, or the given value if the box is empty.
- `map(function)` transforms the contents, if there are any.
- `ifPresent(consumer)` runs the consumer only if there is a value, so here it printed nothing.
- `isPresent()` and `get()` also work, but `get()` on an empty `Optional` throws a `NoSuchElementException`, so always check first.

8. Number Streams

`mapToInt` turns a stream of objects into an `IntStream`, which has number methods of its own:

```
int total = songs.stream().mapToInt(Song::seconds).sum();
OptionalDouble average = songs.stream().mapToInt(Song::seconds).average();
System.out.println(total + " seconds, average " + average.getAsDouble());

IntSummaryStatistics stats = songs.stream().mapToInt(Song::seconds).summaryStatistics()
    ;
System.out.println(stats);
```

```
1793 seconds, average 256.14285714285717
IntSummaryStatistics{count=7, sum=1793, min=176, average=256.142857, max=345}
```

`sum` is never empty; it is 0 for no elements. `average` returns an `OptionalDouble`, because an empty stream has no average; `getAsDouble` takes the value out, and `orElse(0)` would supply a default. `mapToDouble` and `mapToLong` work the same way.

9. Collectors

`collect` is the terminal operation for building a collection or a map, and the `Collectors` class supplies the recipes. The most useful is `groupingBy`, which groups the elements by a key and then does something with each group:

```
Map<String, Long> count = songs.stream()
        .collect(Collectors.groupingBy(Song::artist, TreeMap::new,
                Collectors.counting()));

Map<String, Double> average = songs.stream()
        .collect(Collectors.groupingBy(Song::artist, TreeMap::new,
                Collectors.averagingInt(Song::seconds)));

Map<String, List<String>> titles = songs.stream()
        .collect(Collectors.groupingBy(Song::artist, TreeMap::new,
                Collectors.mapping(Song::title, Collectors.toList())));
```

```
{Ada Vale=2, Neon Harbor=3, The Lanterns=2}
{Ada Vale=316.0, Neon Harbor=247.33333333333334, The Lanterns=209.5}
{Ada Vale=[Northern Lights, Glass Garden], Neon Harbor=[Midnight Drive, Slow Burn, Static],
The Lanterns=[Paper Moons, Echo Park]}
```

`groupingBy` takes up to three arguments: the key, the kind of map to build (here a `TreeMap`, so the artists are sorted), and what to do with each group's elements: count them, average them, or map them and collect the results. The last output is one line, broken to fit the page.

```
Map<Boolean, List<String>> longOrShort = songs.stream()
        .collect(Collectors.partitioningBy(s -> s.seconds() > 240,
                Collectors.mapping(Song::title, Collectors.toList())));

String joined = songs.stream().map(Song::title).collect(Collectors.joining(", "));

Map<String, Integer> longestByArtist = songs.stream()
        .collect(Collectors.toMap(Song::artist, Song::seconds, Math::max,
                TreeMap::new));
```

```
{false=[Paper Moons, Echo Park, Static], true=[Midnight Drive, Slow Burn, Northern Lights,
Glass Garden]}
Midnight Drive, Paper Moons, Slow Burn, Northern Lights, Echo Park, Glass Garden, Static
{Ada Vale=345, Neon Harbor=312, The Lanterns=221}
```

- `partitioningBy` splits the elements into two groups, `true` and `false`.
- `joining` combines strings into one, with a separator.
- `toMap` builds a map from a key and a value for each element. Its third argument says what to do when two elements have the same key; here, keep the larger value. Without it, a repeated key throws an `IllegalStateException`.

10. Laziness and Single Use

Intermediate operations do nothing on their own. Elements flow through the pipeline only when a terminal operation asks for a result, and then only as many as it needs:

```
Stream<String> titles = songs.stream()
        .filter(s -> {
            System.out.println("filter " + s.title());
            return s.seconds() > 300;
        })
        .map(s -> {
            System.out.println("map    " + s.title());
            return s.title();
        });
System.out.println("nothing has run yet");
System.out.println(titles.findFirst().get());
```

```
nothing has run yet
filter Midnight Drive
filter Paper Moons
filter Slow Burn
map    Slow Burn
Slow Burn
```

Nothing printed until `findFirst` ran. Then each song went all the way through the pipeline before the next one started, and once Slow Burn passed, the stream stopped: the last four songs were never examined. This is called *laziness*, and it is why a pipeline with `limit` or `findFirst` can be efficient.

A stream can be used only once. A second terminal operation on the same stream fails:

```
Exception in thread "main" java.lang.IllegalStateException: stream has already been operated
    upon
or closed
```

To run another pipeline, call `stream()` again.

11. Pitfalls

The list from `toList()` cannot be changed

```
List<String> titles = Playlist.songs().stream().map(Song::title).toList();
titles.add("Bonus Track");
```

```
Exception in thread "main" java.lang.UnsupportedOperationException
```

If you need to add to the result, copy it: `new ArrayList<>(titles)`.

Checked exceptions and lambdas

The methods of the functional interfaces do not declare checked exceptions, so a lambda cannot throw one:

```
List<String> contents = names.stream()
        .map(name -> Files.readString(Path.of(name)))
        .toList();
```

```
error: unreported exception IOException; must be caught or declared to be thrown
```

The lambda would have to catch the `IOException` itself, which makes it long and awkward. When the work can throw a checked exception, a plain loop with a `try` block, as in the exceptions notes, is usually clearer.

Changing outside variables

A pipeline should compute its result, not change variables or collections outside it. A lambda cannot change a local variable at all, as the lambda notes showed, and adding to an outside list from `forEach` works but defeats the purpose: use a collector instead.

When a loop is clearer

Streams are best at filtering, transforming, and summarizing. A plain loop is often clearer when each step depends on the one before, when the code must stop in the middle for a complicated reason, or when it handles checked exceptions. Use whichever says what the code does more plainly.

12. Summary

Table 3: Streams in brief.

Idea	Key points
Pipeline	A source, intermediate operations, and one terminal operation. The source never changes.
Intermediate	`filter`, `map`, `sorted`, `distinct`, `limit`, `skip`. Nothing runs until a terminal operation.
Terminal	`toList`, `count`, `forEach`, `anyMatch`, `min`, `max`, `findFirst`, `reduce`, `collect`.
Optional	What `min`, `max`, and `findFirst` return. Use `orElse`, `map`, or `ifPresent`.
Number streams	`mapToInt`, then `sum`, `average`, or `summaryStatistics`.
Collectors	`groupingBy` with `counting`, `averagingInt`, or `mapping`; `partitioningBy`; `joining`; `toMap`.
Rules	One use per stream. `toList()` cannot be changed. No checked exceptions in lambdas.

13. Try It

1. Use a pipeline to find the titles of Neon Harbor's songs, longest first.
2. Find the total seconds for each artist with `groupingBy` and `Collectors.summingInt`.
3. Move the `limit(3)` in Section 5 before `sorted()`. Predict the output, then run it. Why does the order of operations matter?
4. Write a pipeline that prints whether any song is exactly three minutes long, then change it to print the first song under three minutes, or "none" if there is no such song.